

# Docker & Kubernetes — Complete Technical Deep Dive

Reference Document-

<https://medium.com/@deepaksharma2494/understanding-kubernetes-and-docker-easiest-explanation-9505638107e3>

---

## 1. Problem Statement

Modern applications need:

- Consistent environments
- Easy deployment
- Scalability across machines

👉 Solution:

- Docker → Packaging
  - Kubernetes → Orchestration
- 

## 2. What is Docker?

Docker is a containerization platform that:

👉 Packages application + dependencies into a **container**

---

### 2.1 Docker Image vs Container

- Image → Blueprint (immutable)
- Container → Running instance

---

### **Example:**

Image → Java App + JDK + Libraries

Container → Running app from that image

---

## **2.2 Docker Architecture**

- Docker Client
  - Docker Daemon
  - Docker Registry (e.g., Docker Hub)
- 

## **2.3 Dockerfile (Core Concept)**

Defines how image is built

```
FROM openjdk:17
WORKDIR /app
COPY . .
RUN javac Main.java
CMD ["java", "Main"]
```

---

## **2.4 Docker Commands (Important)**

---

### **Build Image**

```
docker build -t my-app .
```

---

### **Run Container**

```
docker run -d -p 8080:8080 my-app
```

---

## List Containers

`docker ps`

---

## Stop Container

`docker stop <container_id>`

---

## Remove Container

`docker rm <container_id>`

---

## View Logs

`docker logs <container_id>`

---

## 2.5 Why Docker?

- Environment consistency
  - “Works on my machine” problem solved
  - Lightweight compared to VMs
- 

## 3. Limitations of Docker

Docker works well on a **single machine**

👉 Problem:

- What if we have 100+ containers?
  - Across multiple servers?
- 

👉 Solution:

Kubernetes

---

## 4. What is Kubernetes?

Kubernetes is a container orchestration system.

👉 It manages:

- Deployment
- Scaling
- Load balancing
- Self-healing

---

## 5. Kubernetes Architecture

---

### 5.1 Master Node (Control Plane)

- API Server
- Scheduler
- Controller Manager

---

### 5.2 Worker Nodes

- Kubelet
- Container Runtime
- Pods

---

## 6. Core Kubernetes Concepts

---

## 6.1 Pod

- Smallest unit in Kubernetes
  - Contains one or more containers
- 

## 6.2 Deployment

- Manages pods
  - Ensures desired state
- 

## 6.3 Service

- Exposes pods
  - Provides stable networking
- 

## 6.4 ReplicaSet

- Ensures number of pods running
- 

## 6.5 Namespace

- Logical separation
- 

# 7. Kubernetes Workflow

1. Developer creates Docker image
2. Pushes image to registry
3. Kubernetes pulls image
4. Creates pods
5. Exposes via service

---

## 8. kubectl Commands (VERY IMPORTANT)

---

### Check Cluster

`kubectl cluster-info`

---

### Get Nodes

`kubectl get nodes`

---

### Get Pods

`kubectl get pods`

---

### Describe Pod

`kubectl describe pod <pod_name>`

---

### Create Deployment

`kubectl create deployment my-app --image=my-app`

---

### Scale Deployment

`kubectl scale deployment my-app --replicas=3`

---

### Expose Service

`kubectl expose deployment my-app --type=NodePort --port=80`

---

## Delete Pod

```
kubectl delete pod <pod_name>
```

---

## Apply YAML

```
kubectl apply -f deployment.yaml
```

---

# 9. YAML Deployment Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: my-app:latest
          ports:
            - containerPort: 8080
```

---

# 10. Key Kubernetes Features

---

## Auto Scaling

- Horizontal Pod Autoscaler (HPA)
- 

## Self-Healing

- Restart failed containers
  - Replace crashed pods
- 

## Load Balancing

- Distributes traffic across pods
- 

## Rolling Updates

- Deploy without downtime
- 

# 11. Docker vs Kubernetes

| Feature    | Docker           | Kubernetes        |
|------------|------------------|-------------------|
| Purpose    | Containerization | Orchestration     |
| Scope      | Single machine   | Multiple machines |
| Scaling    | Manual           | Automatic         |
| Management | Basic            | Advanced          |

---

# 12. Real-World Flow

User Request  
→ Load Balancer



- Kubernetes Service
  - Pod
  - Container
  - Application Response
- 

## 13. Common Mistakes

- ✗ Running containers manually in production
  - ✗ No health checks
  - ✗ Not using replicas
- 

## 14. Production Concepts

- Ingress (external access)
  - ConfigMaps & Secrets
  - Helm (package manager)
  - Monitoring (Prometheus, Grafana)
- 



## Final Insight

Docker solves:

- 👉 Packaging problem

Kubernetes solves:

- 👉 Scaling & management problem
- 



## One Line Summary

Docker = Run container

Kubernetes = Run containers at scale

---

## Interview Line

“Docker ensures consistent runtime environments, while Kubernetes orchestrates containerized workloads with scalability, fault tolerance, and automated management.”